B. New Year and Ascent Sequence

time limit per test 2 seconds
memory limit per test 1024 megabytes
input standard input
output standard output

Rating: 1400 | Tags: binary search, combinatorics, data structures, dp, implementation, sortings |
https://codeforces.com/problemset/problem/1284/B

A sequence $a = [a_1, a_2, \ldots, a_l]$ of length l has an **ascent** if there exists a pair of indices (i, j) such that $1 \le i$ lt $j \le l$ and a_i lt a_j. For example, the sequence [0, 2, 0, 2, 0] has an ascent because of the pair (1, 4), but the sequence [4, 3, 3, 3, 1] doesn't have an ascent.

Let's call a concatenation of sequences p and q the sequence that is obtained by writing down sequences p and q one right after another without changing the order. For example, the concatenation of the [0, 2, 0, 2, 0] and [4, 3, 3, 3, 1] is the sequence [0, 2, 0, 2, 0, 4, 3, 3, 3, 1]. The concatenation of sequences p and q is denoted as $p+q$.

Gyeonggeun thinks that sequences with ascents bring luck. Therefore, he wants to make many such sequences for the new year. Gyeonggeun has n sequences $s_1, s_2, \ldots, s_n$ which may have different lengths.

Gyeonggeun will consider all n^2 pairs of sequences s_x and s_y $(1 \le x, y \le n)$, and will check if its concatenation $s_x + s_y$ has an ascent. Note that he may select the same sequence twice, and the order of selection matters.

Please count the number of pairs (x, y) of sequences $s_1, s_2, \ldots, s_n$ whose concatenation $s_x + s_y$ contains an ascent.

Input

The first line contains the number n $(1 \le n \le 100\,000)$ denoting the number of sequences.

The next n lines contain the number l_i $(1 \le l_i)$ denoting the length of s_i, followed by l_i integers $s_{i,1}, s_{i,2}, \ldots, s_{i,l_i}$ $(0 \le s_{i,j} \le 10^6)$ denoting the sequence s_i.

It is guaranteed that the sum of all l_i does not exceed 100 000.

Output

Print a single integer, the number of pairs of sequences whose concatenation has an ascent.

Examples

Input
5 1 1 1 1 1 2 1 4 1 3

Output
9

Input

```
3
4 2 0 2 0
6 9 9 8 8 7 7
1 6
```

Output

```
7
```

Input

```
10
3 62 24 39
1 17
1 99
1 60
1 64
1 30
2 79 29
2 20 73
2 85 37
1 100
```

Output

```
72
```

Note

For the first example, the following 9 arrays have an ascent: [1, 2], [1, 2], [1, 3], [1, 3], [1, 4], [1, 4], [2, 3], [2, 4], [3, 4]. Arrays with the same contents are counted as their occurences.